

I have a college Vibe Coding lab assignment.

I have forked and cloned the assigned GitHub repo:
SETAPESU26/24_treasure_hunt

My fork is:
DhawalP26/24_treasure_hunt

I have already:

1. Read the README.
2. Cloned the repo.
3. Run the original Python/Pygame game successfully.
4. Recorded the required 10-second BEFORE video.

The assignment says I must modify the EXISTING code, not replace it with an unrelated implementation.

The README requires these 4 features, one by one:

Task 1 — Traps:

Add floor traps that send the player back to their starting position when stepped on.

Done when the player is visibly reset to the start room and a clear status message is shown.

Task 2 — Enemy Guard:

Add a guard that patrols back and forth near the chest.

Touching the guard resets the player to start.

Done when the guard moves between two patrol points each frame and correctly resets the player on collision.

Task 3 — Mini-map:

Draw a small mini-map in a corner showing the dungeon layout.

It must distinguish walls from floor and mark the player's current position.

It must update in real time.

Task 4 — Inventory UI:

Show a HUD inventory slot that is empty initially and displays a key icon after the key is collected.

It must update immediately.

Required testing:

Collect key then chest, trigger every trap type, collide with guard, verify resets, check mini-map against actual grid, verify inventory updates, and test restarting mid-run.

There must be no unnecessary dependencies beyond pygame.

I will provide the current game.py code next.

Do NOT rewrite the entire project.

First understand the existing code and then help me modify it incrementally.

For every change, explain what you're changing and why, then give me the exact code/change to make.

We will test each task before moving to the next.

CURRENT game.py:

```
import pygame
import random

TILE = 40
COLS, ROWS = 20, 15
WALL, FLOOR, CHEST, KEY = 0, 1, 2, 3
SPEED = 3

def generate_world():
    grid = [[WALL]*COLS for _ in range(ROWS)]
    rooms = []
    for _ in range(8):
        w = random.randint(3,6)
        h = random.randint(3,5)
        x = random.randint(1, COLS-w-1)
        y = random.randint(1, ROWS-h-1)
        room = pygame.Rect(x, y, w, h)
        overlap = any(room.inflate(2,2).colliderect(r) for r in rooms)
        if not overlap:
            rooms.append(room)
            for ry in range(y, y+h):
                for rx in range(x, x+w):
                    grid[ry][rx] = FLOOR

    for i in range(len(rooms)-1):
        ax, ay = rooms[i].centerx, rooms[i].centery
        bx, by = rooms[i+1].centerx, rooms[i+1].centery
        cx = ax
        while cx != bx:
            grid[ay][cx] = FLOOR
            cx += 1 if bx > cx else -1
        cy = ay
        while cy != by:
            grid[cy][bx] = FLOOR
            cy += 1 if by > cy else -1

    if len(rooms) >= 2:
        cr, ck = rooms[-1], rooms[-2]
        grid[cr.centery][cr.centerx] = CHEST
        grid[ck.centery][ck.centerx] = KEY

    start = rooms[0] if rooms else None
    return grid, start

COLORS = {
    WALL: (60,50,70),
    FLOOR: (200,190,170),
    CHEST: (200,160,30),
    KEY: (220,220,60),
}

class Player:
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (60,120,220)
        self.has_key = False

    def move(self, keys, grid, rows, cols):
```

```

        dx=dy=0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx=-SPEED
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx=SPEED
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy=-SPEED
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy=SPEED
        self._try_move(dx,0,grid,rows,cols)
        self._try_move(0,dy,grid,rows,cols)

    def _try_move(self, dx, dy, grid, rows, cols):
        new = self.rect.move(dx,dy)
        for px,py in [(new.left,new.top),(new.right-1,new.top),(new.left,new.bottom-1),(new.right-1,new.bottom-1)]:
            c,r=px//TILE,py//TILE
            if not(0<=r<rows and 0<=c<cols) or grid[r][c]==WALL:
                return
        self.rect=new

    def draw(self, screen):
        pygame.draw.ellipse(screen, self.color, self.rect)
        if self.has_key:
            pygame.draw.circle(screen, (220,220,60), (self.rect.right-6, self.rect.top+6), 5)

WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH, HEIGHT))
        pygame.display.set_caption("Treasure Hunt")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 24)
        self.big_font = pygame.font.SysFont("monospace", 40, bold=True)
        self.reset()

    def reset(self):
        self.grid, start = generate_world()
        if start:
            sx = start.x * TILE + 6
            sy = start.y * TILE + 6
        else:
            sx, sy = TILE+6, TILE+6
        self.player = Player(sx, sy)
        self.won = False
        self.status = "Find the KEY, then the CHEST!"

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r: self.reset()
        return True

    def update(self):
        if self.won: return
        keys = pygame.key.get_pressed()
        self.player.move(keys, self.grid, ROWS, COLS)
        pr = self.player.rect.centery // TILE
        pc = self.player.rect.centerx // TILE
        if 0<=pr<ROWS and 0<=pc<COLS:
            cell = self.grid[pr][pc]
            if cell == KEY:
                self.player.has_key = True
                self.grid[pr][pc] = FLOOR
                self.status = "Got the key! Find the CHEST!"
            elif cell == CHEST and self.player.has_key:
                self.won = True
                self.status = "Treasure found!"

    def draw(self):
        self.screen.fill((30,25,40))
        for r in range(ROWS):
            for c in range(COLS):
                cell = self.grid[r][c]
                rect = pygame.Rect(c*TILE, r*TILE, TILE, TILE)
                pygame.draw.rect(self.screen, COLORS[cell], rect)
                if cell == KEY:
                    pygame.draw.circle(self.screen, (255,240,60),(c*TILE+TILE//2, r*TILE+TILE//2),10)
                elif cell == CHEST:
                    pygame.draw.rect(self.screen,(180,120,20),rect.inflate(-12,-12),border_radius=4)
        self.player.draw(self.screen)
        hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
        pygame.draw.rect(self.screen,(20,20,35),hud)
        st = self.font.render(self.status+" | R=Restart", True, (200,200,200))
        self.screen.blit(st,(8,ROWS*TILE+13))
        if self.won:
            ov=pygame.Surface((WIDTH,ROWS*TILE),pygame.SRCALPHA)
            ov.fill((0,0,0,140))
            self.screen.blit(ov,(0,0))
            msg=self.big_font.render("TREASURE FOUND!", True,(220,180,30))
            sub=self.font.render("Press R to Play Again",True,(180,180,180))
            self.screen.blit(msg,(WIDTH//2-msg.get_width()//2,ROWS*TILE//2-30))
            self.screen.blit(sub,(WIDTH//2-sub.get_width()//2,ROWS*TILE//2+20))
        pygame.display.flip()

    def run(self):
        running=True
        while running:
            running=self.handle_events()
            self.update()
            self.draw()
            self.clock.tick(FPS)
        pygame.quit()

if __name__ == "__main__":

```

```
engine = GameEngine()  
engine.run()
```

👍 Got it. We'll treat the existing `game.py` as the base and modify it incrementally — **no rewrite, no unnecessary architecture changes, no extra dependencies.**

The order will be:

1. **Task 1 — Traps** → implement + test
2. **Task 2 — Guard** → implement + test
3. **Task 3 — Mini-map** → implement + test
4. **Task 4 — Inventory UI** → implement + test
5. **Final testing checklist** → make sure every rubric requirement works
6. Then we can deal with the **AFTER video / Git commit / submission** if needed.

First: I've understood the existing structure

Your current game already has some useful pieces we should **reuse**, rather than rebuilding:

- `generate_world()` → procedural dungeon generation
- `grid` → authoritative dungeon layout
- `Player` → movement + collision with walls
- `GameEngine.reset()` → perfect place to initialize/reset new game state
- `GameEngine.update()` → perfect place for gameplay interactions
- `GameEngine.draw()` → perfect place for visual features/HUD
- `handle_events()` → already handles R restart

So we're going to **extend these**, not replace them.

One important thing I noticed: the dungeon is generated randomly every time `reset()` is called. That's fine, but it means our traps/guard need to be placed **after the world is generated**, and their positions need to remain valid for that generated grid.

Task 1 — Traps

Let's do **only Task 1** right now.

The cleanest approach is to introduce a new grid cell type:

Python

Run

Then we'll:

- generate several traps on existing `FLOOR` tiles,
- draw them visibly,
- remember the player's starting pixel position,
- detect when the player steps onto a trap,
- teleport him back to the starting position,
- show a clear status message such as:

"Ouch! Trap triggered! Back to START!"

Why this approach?

Because the existing game already represents the dungeon using:

Python

Run

So traps being another grid cell type fits the existing design instead of creating some weird parallel trap system.

Change 1 — Add `TRAP`

At the top, change:

Python

Run

to:

Python

Run

Then add a trap color to `COLORS` :

Python

Run

So this section becomes:

```
COLORS = {  
    WALL: (60,50,70),  
    FLOOR: (200,190,170),  
    CHEST: (200,160,30),  
    KEY: (220,220,60),  
    TRAP: (180,50,50),  
}
```

Change 2 — Generate traps

Inside `GameEngine.reset()` , we currently have:

```
def reset(self):  
    self.grid, start = generate_world()  
    if start:  
        sx = start.x * TILE + 6  
        sy = start.y * TILE + 6  
    else:  
        sx, sy = TILE+6, TILE+6  
    self.player = Player(sx, sy)  
    self.won = False  
    self.status = "Find the KEY, then the CHEST!"
```

We're going to add trap generation *after* the player starting position is known.

Replace that whole method with:

```

def reset(self):
    self.grid, start = generate_world()

    if start:
        sx = start.x * TILE + 6
        sy = start.y * TILE + 6
    else:
        sx, sy = TILE+6, TILE+6

    self.start_pos = (sx, sy)
    self.player = Player(sx, sy)

    # Place traps on floor tiles, away from the starting position.
    trap_count = 4
    placed = 0

    while placed < trap_count:
        r = random.randint(0, ROWS - 1)
        c = random.randint(0, COLS - 1)

        if self.grid[r][c] == FLOOR:
            if not (start and start.collidepoint(c, r)):
                self.grid[r][c] = TRAP
                placed += 1

    self.won = False
    self.status = "Find the KEY, then the CHEST!"

```

One small but important detail

This:

Python

Run

keeps traps out of the starting room.

Otherwise you could spawn into a trap and instantly trigger it, which would be kinda dumb for a lab assignment. 🤖

Change 3 — Detect a trap

Now go into `update()` .

Currently you have:

```

if 0<=pr<ROWS and 0<=pc<COLS:
    cell = self.grid[pr][pc]
    if cell == KEY:
        ...
    elif cell == CHEST and self.player.has_key:
        ...

```

Add the trap condition **before** the key/chest conditions:

```

if 0<=pr<ROWS and 0<=pc<COLS:
    cell = self.grid[pr][pc]

    if cell == TRAP:
        self.player.rect.topleft = self.start_pos
        self.status = "Ouch! Trap triggered! Back to START!"

    elif cell == KEY:
        self.player.has_key = True
        self.grid[pr][pc] = FLOOR
        self.status = "Got the key! Find the CHEST!"

    elif cell == CHEST and self.player.has_key:
        self.won = True
        self.status = "Treasure found!"

```

Why topleft?

We saved:

Python

Run

which is the player's original pixel position.

So:

Python

Run

puts the player **exactly where they spawned**, rather than merely putting them somewhere inside the starting room.

Change 4 — Make traps visually obvious

Your existing `draw()` already automatically draws every grid cell using:

Python

Run

So simply adding:

Python

Run

already makes trap tiles red.

But I'd make them **more visibly identifiable as traps**, because the rubric says the trap should be apparent.

Inside your drawing loop, after the key/chest conditions, add:

```
elif cell == TRAP:
    pygame.draw.line(
        self.screen,
        (80,20,20),
        (c*TILE+8, r*TILE+8),
        (c*TILE+TILE-8, r*TILE+TILE-8),
        4
    )
    pygame.draw.line(
        self.screen,
        (80,20,20),
        (c*TILE+TILE-8, r*TILE+8),
        (c*TILE+8, r*TILE+TILE-8),
        4
    )
```

So this part:

```
if cell == KEY:
    pygame.draw.circle(...)
elif cell == CHEST:
    pygame.draw.rect(...)
```

becomes:

```

if cell == KEY:
    pygame.draw.circle(
        self.screen,
        (255,240,60),
        (c*TILE+TILE//2, r*TILE+TILE//2),
        10
    )

elif cell == CHEST:
    pygame.draw.rect(
        self.screen,
        (180,120,20),
        rect.inflate(-12,-12),
        border_radius=4
    )

elif cell == TRAP:
    pygame.draw.line(
        self.screen,
        (80,20,20),
        (c*TILE+8, r*TILE+8),
        (c*TILE+TILE-8, r*TILE+TILE-8),
        4
    )
    pygame.draw.line(
        self.screen,
        (80,20,20),
        (c*TILE+TILE-8, r*TILE+8),
        (c*TILE+8, r*TILE+TILE-8),
        4
    )

```

That's Task 1

Don't touch Tasks 2–4 yet.

Run:

Bash

Then verify these specifically:

Test A — Normal gameplay

- Game starts normally.
- Player isn't spawned on a trap.
- You can move normally.

Test B — Trap

Walk onto a red trap.

Expected:

Player immediately goes back to the starting position.

And bottom HUD should say:

Ouch! Trap triggered! Back to START!

Test C — Restart

Press `R`.

Expected:

- new dungeon generated
- player gets a new starting position
- traps regenerate
- no crash

Test D — Existing functionality

Make sure:

- key can still be collected
- chest still works after collecting key
- `R` still restarts

Don't move to Task 2 until these work.

If you hit an error, paste the error + your current `game.py`. I'll debug **that exact version** rather than giving you some disconnected replacement.



Task 1 is implemented and tested successfully.

Now implement Task 2 — Enemy Guard.

Add a guard near the chest that continuously patrols back and forth between two patrol points each frame.

If the player touches the guard, reset the player to the starting position.

Keep the existing game structure and mechanics intact. Do not rewrite unrelated code.

Explain the changes clearly and give me the exact code changes.

Afterward, give me specific steps to test:

1. guard patrol movement
2. player-guard collision
3. player reset after collision
4. restarting with R

Show less ^



Yep. Task 2 should be a **small extension of the existing code**, not a new enemy system.

The cleanest implementation is:

- Create a `Guard` class.
- Place it on a floor tile near the chest.
- Give it two patrol points.
- Move it every frame between those points.
- Use `pygame.Rect.colliderect()` for player collision.
- Reuse your existing `self.start_pos` to reset the player.
- Reinitialize the guard inside `reset()`, so `R` works automatically.

1. Add the `Guard` class

Put this **after the** `Player` class, before `WIDTH = ...`.

Python

Run

```
class Guard:
    def __init__(self, x, y, patrol_start, patrol_end):
        self.rect = pygame.Rect(x, y, 28, 28)
        self.color = (180, 60, 60)
```

```

self.patrol_start = patrol_start
self.patrol_end = patrol_end
self.target = patrol_end

self.speed = 2

def update(self):
    target_x, target_y = self.target

    dx = target_x - self.rect.x
    dy = target_y - self.rect.y

    if abs(dx) <= self.speed and abs(dy) <= self.speed:
        self.rect.topleft = self.target

        if self.target == self.patrol_end:
            self.target = self.patrol_start
        else:
            self.target = self.patrol_end
    else:
        if dx != 0:
            self.rect.x += self.speed if dx > 0 else -self.speed

        if dy != 0:
            self.rect.y += self.speed if dy > 0 else -self.speed

def draw(self, screen):
    pygame.draw.rect(screen, self.color, self.rect, border_radius=6)
    pygame.draw.circle(
        screen,
        (255, 220, 220),
        (self.rect.centerx - 5, self.rect.centery - 4),
        3
    )
    pygame.draw.circle(
        screen,
        (255, 220, 220),
        (self.rect.centerx + 5, self.rect.centery - 4),
        3
    )

```

What this does

The guard has:

Python

Run

```

self.patrol_start
self.patrol_end

```

These are the two positions it moves between.

Every frame:

Python

Run

```
self.guard.update()
```

moves it toward its current target.

When it reaches the target, this:

Python

Run

```
if self.target == self.patrol_end:
    self.target = self.patrol_start
else:
    self.target = self.patrol_end
```

reverses its direction.

So the movement becomes:

Point A → Point B → Point A → Point B → ...

continuously.

2. Create the guard in `reset()`

Your current `reset()` already generates the dungeon and traps.

After your trap-generation code, add the guard setup.

You should currently have something roughly like:

Python

Run

```
self.won = False
self.status = "Find the KEY, then the CHEST!"
```

Before those two lines, add:

Python

Run

```
# Create a guard near the chest.
chest_pos = None

for r in range(ROWS):
    for c in range(COLS):
```

```

        if self.grid[r][c] == CHEST:
            chest_pos = (c, r)
            break
    if chest_pos:
        break

    if chest_pos:
        chest_c, chest_r = chest_pos

        guard_x = chest_c * TILE + 6
        guard_y = chest_r * TILE + 6

        patrol_start = (guard_x - TILE, guard_y)
        patrol_end = (guard_x + TILE, guard_y)

        self.guard = Guard(
            guard_x,
            guard_y,
            patrol_start,
            patrol_end
        )
    else:
        self.guard = None

```

So the relevant portion of `reset()` becomes:

Python

Run

```

# Place traps on floor tiles, away from the starting position.
trap_count = 4
placed = 0

while placed < trap_count:
    r = random.randint(0, ROWS - 1)
    c = random.randint(0, COLS - 1)

    if self.grid[r][c] == FLOOR:
        if not (start and start.collidepoint(c, r)):
            self.grid[r][c] = TRAP
            placed += 1

# Create a guard near the chest.
chest_pos = None

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == CHEST:
            chest_pos = (c, r)
            break
    if chest_pos:
        break

```

```

if chest_pos:
    chest_c, chest_r = chest_pos

    guard_x = chest_c * TILE + 6
    guard_y = chest_r * TILE + 6

    patrol_start = (guard_x - TILE, guard_y)
    patrol_end = (guard_x + TILE, guard_y)

    self.guard = Guard(
        guard_x,
        guard_y,
        patrol_start,
        patrol_end
    )
else:
    self.guard = None

self.won = False
self.status = "Find the KEY, then the CHEST!"

```

One important thing

The guard is placed on the **same tile as the chest** initially, then patrols one tile left and one tile right.

That's deliberate: it makes the guard clearly "near the chest" without creating another complicated placement algorithm.

3. Update the guard every frame

Now go to:

Python

Run

```
def update(self):
```

You currently have:

Python

Run

```

if self.won: return
keys = pygame.key.get_pressed()
self.player.move(keys, self.grid, ROWS, COLS)

```

Change it to:

Python

Run

```
if self.won:
    return

keys = pygame.key.get_pressed()
self.player.move(keys, self.grid, ROWS, COLS)

if self.guard:
    self.guard.update()

    if self.player.rect.colliderect(self.guard.rect):
        self.player.rect.topleft = self.start_pos
        self.status = "Guard caught you! Back to START!"
```

Why `colliderect()` ?

You already have:

Python

Run

```
self.player.rect
```

and the guard also has a `pygame.Rect` .

So:

Python

Run

```
self.player.rect.colliderect(self.guard.rect)
```

checks whether their rectangles overlap.

No complicated distance calculations needed.

4. Draw the guard

In `draw()` , you currently have:

Python

Run

```
self.player.draw(self.screen)
```

Add the guard immediately after it:

Python

Run

```
self.player.draw(self.screen)

if self.guard:
    self.guard.draw(self.screen)
```

So the player and guard are both rendered before the HUD.



Your Task 2 changes, conceptually

Your existing architecture now looks like:

```
GameEngine
|
├─ grid
├─ player
├─ guard
|
├─ reset()
|   ├─ generate world
|   ├─ place traps
|   └─ create guard
|
├─ update()
|   ├─ move player
|   ├─ move guard
|   ├─ check guard collision
|   └─ check key/chest
|
└─ draw()
    ├─ draw dungeon
    ├─ draw player
    ├─ draw guard
    └─ draw HUD
```

That's exactly the kind of incremental modification we want here. We're not touching your dungeon-generation logic or rewriting the player mechanics.



Task 2 testing

Run:

Bash

```
python game.py
```

1. Test guard patrol movement

Go near the chest and watch the guard.

You should see it move:

left → right → left → right

continuously.

It should **not teleport** between the two points.

Also let the game sit untouched for several seconds. The guard should keep patrolling.

2. Test player-guard collision

Walk directly into the guard.

Expected:

```
Player touches guard  
    ↓  
Collision detected
```

The guard itself should remain active and continue patrolling.

3. Test player reset

After touching the guard, the player should immediately return to the exact starting position.

The HUD should say:

Guard caught you! Back to START!

Then move around again and verify that the game isn't frozen.

You should be able to encounter the guard again.

4. Test R restart

Press:

R

Verify:

- new dungeon is generated
- player gets reset
- traps are regenerated
- guard is recreated
- guard starts its patrol again
- status returns to:

Find the KEY, then the CHEST!

Then let the guard patrol for a few seconds and confirm it works again.

One thing to watch for

Because your dungeon is procedurally generated, **don't assume the guard will always have a perfect-looking open corridor around the chest**. With the current implementation it patrols one tile left/right from the chest, which is normally fine because the chest is placed in a room, but if you encounter a weird generated layout where the guard overlaps a wall, tell me.

Test Task 2 completely before we touch Task 3. If you get an error or the guard behaves weirdly, paste your current `game.py` and we'll fix that version rather than piling more changes on top.



Absolutely. Task 3 is pretty clean because your existing `grid` already contains everything the mini-map needs. We **do not need another dungeon representation**.

We'll add one method to `GameEngine` that draws a scaled-down version of `self.grid`, then call it from the existing `draw()` method.

What we're adding

The actual dungeon is:

```
20 columns × 15 rows
```

At `TILE = 40`, that's `800 × 600`, so obviously we don't want another `800×600` map on top of the game. 😄

We'll make each mini-map cell about **8×8 pixels**:

```
20 × 8 = 160 px wide
15 × 8 = 120 px tall
```

We'll put it in the **top-right corner** with a small border.

The mini-map will use:

- dark color → wall
- light color → floor
- gold → chest
- yellow → key
- red → trap
- blue → player

And because it reads directly from:

Python

```
self.grid
```

Run

it automatically represents the current dungeon.

Change 1 — Add mini-map constants

Near your existing constants:

Python

Run

```
WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60
```

add:

Python

Run

```
MINIMAP_TILE = 8
MINIMAP_MARGIN = 10
```

So:

Python

Run

```
WIDTH = COLS * TILE
HEIGHT = ROWS * TILE + 50
FPS = 60
```

```
MINIMAP_TILE = 8
MINIMAP_MARGIN = 10
```

Why?

`MINIMAP_TILE` controls how large each dungeon cell appears on the mini-map.

With:

Python

Run

```
MINIMAP_TILE = 8
```

the entire 20×15 dungeon becomes:

```
160 × 120 pixels
```

which fits nicely in the corner.

Change 2 — Add a `draw_minimap()` method

Inside `GameEngine`, add this method **before** `draw()`:

Python

Run

```
def draw_minimap(self):
    map_width = COLS * MINIMAP_TILE
    map_height = ROWS * MINIMAP_TILE

    x0 = WIDTH - map_width - MINIMAP_MARGIN
    y0 = MINIMAP_MARGIN

    # Background and border
    pygame.draw.rect(
        self.screen,
        (10, 10, 15),
        (x0 - 3, y0 - 3, map_width + 6, map_height + 6)
    )

    # Draw dungeon cells
    for r in range(ROWS):
        for c in range(COLS):
            cell = self.grid[r][c]

            if cell == WALL:
                color = (45, 40, 55)
            else:
                color = (190, 180, 160)

            rect = pygame.Rect(
                x0 + c * MINIMAP_TILE,
                y0 + r * MINIMAP_TILE,
                MINIMAP_TILE,
                MINIMAP_TILE
            )

            pygame.draw.rect(self.screen, color, rect)

    # Draw the key
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] == KEY:
                pygame.draw.rect(
                    self.screen,
                    (255, 230, 60),
                    (
                        x0 + c * MINIMAP_TILE,
                        y0 + r * MINIMAP_TILE,
                        MINIMAP_TILE,
                        MINIMAP_TILE
                    )
                )

    # Draw the chest
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] == CHEST:
```

```

        pygame.draw.rect(
            self.screen,
            (180, 120, 20),
            (
                x0 + c * MINIMAP_TILE,
                y0 + r * MINIMAP_TILE,
                MINIMAP_TILE,
                MINIMAP_TILE
            )
        )

# Draw traps
for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == TRAP:
            pygame.draw.rect(
                self.screen,
                (180, 50, 50),
                (
                    x0 + c * MINIMAP_TILE,
                    y0 + r * MINIMAP_TILE,
                    MINIMAP_TILE,
                    MINIMAP_TILE
                )
            )

# Draw player position
player_col = self.player.rect.centerx // TILE
player_row = self.player.rect.centery // TILE

if 0 <= player_row < ROWS and 0 <= player_col < COLS:
    pygame.draw.rect(
        self.screen,
        (50, 120, 255),
        (
            x0 + player_col * MINIMAP_TILE,
            y0 + player_row * MINIMAP_TILE,
            MINIMAP_TILE,
            MINIMAP_TILE
        )
    )

```

What this method is doing

The important part is this:

Python

Run

```
cell = self.grid[r][c]
```

We're directly reading your **existing** dungeon grid.

For every cell:

Python

Run

```
if cell == WALL:
    color = (45, 40, 55)
else:
    color = (190, 180, 160)
```

So the map preserves the actual wall/floor structure.

Then we separately draw the special objects:

- KEY
- CHEST
- TRAP

Finally, we calculate the player's current grid position:

Python

Run

```
player_col = self.player.rect.centerx // TILE
player_row = self.player.rect.centery // TILE
```

That's important because your player moves in **pixels**, while the dungeon exists in **grid cells**.

For example, if the player's center is at:

```
x = 245
y = 367
```

then:

Python

Run

```
245 // 40 = 6
367 // 40 = 9
```

So the player is currently in:

```
grid[9][6]
```

The mini-map then highlights that exact cell.

Change 3 — Call the mini-map from `draw()`

At the end of your dungeon/player drawing section, you currently have something like:

Python

Run

```
self.player.draw(self.screen)

if self.guard:
    self.guard.draw(self.screen)
```

Immediately **after the guard**, add:

Python

Run

```
self.draw_minimap()
```

So it becomes:

Python

Run

```
self.player.draw(self.screen)

if self.guard:
    self.guard.draw(self.screen)

self.draw_minimap()
```

Why here?

Because `draw_minimap()` needs:

Python

Run

```
self.grid
self.player
```

which are already available.

And because `draw()` runs every frame, the mini-map also gets redrawn every frame.

That gives us the required **real-time update**.

One small optimization

The method above loops through the grid separately for walls/floor, key, chest, and traps.

That is completely fine for a $20 \times 15 = 300$ -cell dungeon, but we can make it cleaner by doing everything in one loop.

I'd actually recommend this version instead.

Replace the `draw_minimap()` method above with:

Python

Run

```
def draw_minimap(self):
    map_width = COLS * MINIMAP_TILE
    map_height = ROWS * MINIMAP_TILE

    x0 = WIDTH - map_width - MINIMAP_MARGIN
    y0 = MINIMAP_MARGIN

    # Background and border
    pygame.draw.rect(
        self.screen,
        (10, 10, 15),
        (x0 - 3, y0 - 3, map_width + 6, map_height + 6)
    )

    # Draw dungeon layout directly from the existing grid
    for r in range(ROWS):
        for c in range(COLS):
            cell = self.grid[r][c]

            if cell == WALL:
                color = (45, 40, 55)
            elif cell == FLOOR:
                color = (190, 180, 160)
            elif cell == KEY:
                color = (255, 230, 60)
            elif cell == CHEST:
                color = (180, 120, 20)
            elif cell == TRAP:
                color = (180, 50, 50)
            else:
                color = (45, 40, 55)

            rect = pygame.Rect(
                x0 + c * MINIMAP_TILE,
                y0 + r * MINIMAP_TILE,
                MINIMAP_TILE,
                MINIMAP_TILE
            )
```

```

pygame.draw.rect(self.screen, color, rect)

# Draw player position
player_col = self.player.rect.centerx // TILE
player_row = self.player.rect.centery // TILE

if 0 <= player_row < ROWS and 0 <= player_col < COLS:
    pygame.draw.rect(
        self.screen,
        (50, 120, 255),
        (
            x0 + player_col * MINIMAP_TILE,
            y0 + player_row * MINIMAP_TILE,
            MINIMAP_TILE,
            MINIMAP_TILE
        )
    )

```

Use this version. It's simpler and does the same job.

Final structure

You should now have:

Python

Run

```

class GameEngine:
    def __init__(self):
        ...

    def reset(self):
        ...

    def handle_events(self):
        ...

    def update(self):
        ...

    def draw_minimap(self):
        ...

    def draw(self):
        ...

    def run(self):
        ...

```

No other game mechanics need to change.



Task 3 testing

Now test this systematically.

Test 1 — Layout accuracy

Start the game and compare the actual dungeon with the mini-map.

Look specifically at:

- rooms
- corridors
- walls
- dead ends

For example, if the actual game has:



the mini-map should have the **same structure**, just scaled down.

Because we're using:

Python

```
self.grid[r][c]
```

Run

there shouldn't be any discrepancy.

Test 2 — Player position

Move the player around slowly.

Watch the blue mini-map square.

It should move from cell to cell as the player moves.

Walk:

up
down
left
right

and make sure the blue marker corresponds to the player's actual grid location.

Test 3 — Corridors

Walk through a narrow corridor.

Verify that the mini-map shows:

- walls on the correct sides
- floor in the corridor
- player marker inside the corridor

This is a particularly good test because incorrect row/column calculations become obvious here.

Test 4 — Traps

Find a trap.

The mini-map should show the trap as a **red cell**.

Step on it.

After the reset, the blue player marker should immediately jump back to the starting cell.

This also verifies that the mini-map is actually being redrawn after gameplay changes.

Test 5 — Key

Find the key.

Before collecting:

- key should appear as a yellow mini-map cell.

After collecting:

Python

Run

```
self.grid[pr][pc] = FLOOR
```

changes that grid cell from KEY → FLOOR .

Therefore the mini-map should immediately change that cell from yellow → floor color.

That's a nice test that proves the mini-map is using the **actual grid**, rather than a stale copy.

Test 6 — Chest

Check that the chest appears at the same location on:

- the actual dungeon
- the mini-map

Then collect the key and reach the chest.

The existing victory behavior should remain unchanged.

Test 7 — Restart

Press:

R

The dungeon is regenerated.

Verify that the mini-map also changes to match the **new dungeon layout** and the player marker returns to the new starting position.

This is important because your world is procedural.

Task 3 acceptance checklist

Before moving to Task 4, you should be able to tick all of these:

- ☐ Mini-map visible in a corner
- ☐ Walls distinguishable from floor
- ☐ Player position shown
- ☐ Player marker updates while moving
- ☐ Key/chest/traps represented correctly
- ☐ Key disappears/changes after collection

- ☐ Trap reset reflected immediately
- ☐ New dungeon after R produces a matching new mini-map
- ☐ No external dependencies added
- ☐ Existing gameplay still works

Once all of those pass, **Task 4 — Inventory UI** is the next isolated change.



Tasks 1, 2, and 3 are implemented and tested successfully.

Now implement Task 4 — Inventory UI.

Add a HUD inventory slot that:

- is empty when the game starts
- displays a key icon once the player collects the key
- updates immediately when the key is collected
- remains correct after traps, guard collisions, and restarting

Use the existing `player.has_key` state where appropriate.

Do not rewrite unrelated code or add external dependencies.

Explain the changes and give me the exact code changes.

Then give me specific tests for both the empty and key-collected states, including restarting mid-run.

Show less ^

Yep — Task 4 is the simplest one because you already have the exact state we need:

Python

Run

```
self.player.has_key
```

Your `Player` class already sets:

Python

Run

```
self.has_key = False
```

and your existing key logic already changes it to:

Python

Run

```
self.player.has_key = True
```

So we **don't need another inventory system**. We just need to expose that state visually in the HUD.

Task 4 — Inventory UI

We'll add a small **INVENTORY** box to the existing bottom HUD.

Conceptually:

Before key:

Find the KEY, then the CHEST! INVENTORY []

After key:

Got the key! Find the CHEST! INVENTORY [🔑]

Since we shouldn't rely on external images/fonts, we'll draw the key icon ourselves using basic Pygame shapes.

Change 1 — Add an inventory drawing method

Inside `GameEngine`, add this method **before** `draw()`:

Python

Run

```
def draw_inventory(self):
    inventory_x = WIDTH - 150
    inventory_y = ROWS * TILE + 7

    # Inventory label
    label = self.font.render("INV", True, (200, 200, 200))
    self.screen.blit(label, (inventory_x, inventory_y + 5))

    # Inventory slot
    slot_x = inventory_x + 45
    slot_y = inventory_y
```

```
slot_size = 36

pygame.draw.rect(
    self.screen,
    (60, 60, 75),
    (slot_x, slot_y, slot_size, slot_size),
    border_radius=4
)

pygame.draw.rect(
    self.screen,
    (150, 150, 160),
    (slot_x, slot_y, slot_size, slot_size),
    2,
    border_radius=4
)

# Draw key only after it has been collected
if self.player.has_key:
    center_x = slot_x + slot_size // 2
    center_y = slot_y + slot_size // 2

    # Key ring
    pygame.draw.circle(
        self.screen,
        (255, 230, 60),
        (center_x - 6, center_y - 4),
        6,
        2
    )

    # Key shaft
    pygame.draw.line(
        self.screen,
        (255, 230, 60),
        (center_x, center_y - 4),
        (center_x + 11, center_y - 4),
        3
    )

    # Key teeth
    pygame.draw.line(
        self.screen,
        (255, 230, 60),
        (center_x + 6, center_y - 4),
        (center_x + 6, center_y + 2),
        3
    )

    pygame.draw.line(
        self.screen,
        (255, 230, 60),
```

```

        (center_x + 11, center_y - 4),
        (center_x + 11, center_y + 2),
        3
    )

```

What this does

The inventory slot is **always visible**.

But the key itself is conditional:

Python

Run

```
if self.player.has_key:
```

So:

```
has_key = False
    ↓
empty slot
```

and:

```
has_key = True
    ↓
key icon appears
```

That's exactly what the assignment requires.

Change 2 — Call it from draw()

You currently have your HUD code near the bottom of draw() :

Python

Run

```

hud = pygame.Rect(0,ROWS*TILE,WIDTH,50)
pygame.draw.rect(self.screen,(20,20,35),hud)

st = self.font.render(
    self.status+" | R=Restart",
    True,
    (200,200,200)
)

self.screen.blit(st,(8,ROWS*TILE+13))

```

Immediately after:

Python

Run

```
self.screen.blit(st, (8, ROWS*TILE+13))
```

add:

Python

Run

```
self.draw_inventory()
```

So it becomes:

Python

Run

```
hud = pygame.Rect(0, ROWS*TILE, WIDTH, 50)
pygame.draw.rect(self.screen, (20, 20, 35), hud)

st = self.font.render(
    self.status+" | R=Restart",
    True,
    (200, 200, 200)
)

self.screen.blit(st, (8, ROWS*TILE+13))

self.draw_inventory()
```

Why this updates immediately

Your game runs:

```
handle_events()
  ↓
update()
  ↓
draw()
  ↓
repeat every frame
```

When the player touches the key, your existing code does:

Python

Run

```

if cell == KEY:
    self.player.has_key = True
    self.grid[pr][pc] = FLOOR

```

Then, **in that same frame**, `draw()` runs.

`draw_inventory()` checks:

Python

Run

```
self.player.has_key
```

which is now `True` .

Therefore the icon appears immediately.

No extra inventory variable is needed.

What happens with traps and guard collisions?

This is actually a useful consequence of using `player.has_key` .

When a trap triggers, we currently do:

Python

Run

```
self.player.rect.topleft = self.start_pos
```

We **do not** do:

Python

Run

```
self.player.has_key = False
```

Therefore:

```

Collect key
↓
has_key = True
↓
Hit trap
↓
Player returns to start
↓
has_key is STILL True

```

↓
Inventory still shows key

That's probably what we want: the trap resets the player's **position**, not their inventory.

Same thing with the guard:

Collect key
↓
has_key = True
↓
Guard collision
↓
Player returns to start
↓
Key remains in inventory

The assignment only says collisions should reset the player to start; it doesn't say they should remove the key.

What happens when pressing R ?

This is different.

Your `reset()` creates a completely new player:

Python

```
self.player = Player(sx, sy)
```

Run

and the `Player` constructor does:

Python

```
self.has_key = False
```

Run

So:

Press R
↓
new Player
↓
has_key = False
↓
inventory becomes empty

Exactly what we want.

Task 4 Testing

Test 1 — Empty inventory at startup

Start the game.

At the bottom HUD you should see something like:

INV []

The slot should be empty.

There should be **no key icon** inside it.

Test 2 — Collect the key

Find the key and walk onto it.

Immediately verify:

INV [🔑]

The key icon should appear **without restarting or needing another action**.

Also verify that the key disappears from the dungeon itself.

Test 3 — Key remains after trap

After collecting the key, deliberately step on a trap.

Expected:

Player → START

Inventory → STILL HAS KEY

So the player should return to the start, but the inventory should still display the key.

Test 4 — Key remains after guard collision

Again, after collecting the key:

1. Find the guard.
2. Touch the guard.
3. Player should reset to start.
4. Check the inventory.

Expected:

[]

The key should **still be there**.

Test 5 — Restart after collecting key

This one is important.

Do:

```
Start
↓
Collect key
↓
Inventory shows key
↓
Press R
```

Expected:

```
New dungeon
New player
Inventory EMPTY
```

The key should disappear from the inventory because `reset()` creates a new `Player` .

Test 6 — Restart before collecting key

Start a fresh game and **don't collect the key**.

Press `R` .

Expected:

```
Inventory remains empty
```

Then collect the key and verify that it appears again.

Final Task 4 checklist

Before considering the entire lab complete:

- ☐ Inventory slot visible at startup
- ☐ Slot is empty initially
- ☐ Key icon appears immediately after collecting key
- ☐ Key icon remains after trap reset
- ☐ Key icon remains after guard collision
- ☐ R clears the inventory
- ☐ New key can be collected after restarting
- ☐ No external dependencies added
- ☐ Existing key/chest mechanics still work

One thing I'd **not** change: don't create something like `self.inventory = []` just for this. That would duplicate state you already have. `player.has_key` is the source of truth, and the HUD should simply reflect it.



The four tasks are now implemented and the game is working.

I found one edge-case during testing: because the dungeon and traps are generated randomly, some generated layouts can become effectively unwinnable. A trap can occupy a critical corridor or doorway, making it impossible to reach the key or chest without triggering the trap and being sent back to the start.

I do NOT want to rewrite the project.

Please inspect the current code and make the smallest reasonable change that guarantees the generated dungeon remains playable:

- the starting position must be reachable,
- the key must be reachable from the start,
- the chest must be reachable after collecting the key,
- traps should still be present,

- traps should not simply be removed,
- the existing trap behavior should remain unchanged,
- the guard, mini-map, and inventory must continue working,
- R restart must continue generating a new dungeon.

Use the existing grid representation.

Please explain the problem first, then give the minimal code modification. Afterward give me a testing procedure to verify this over multiple random restarts.

Show less ^

Worked for 10s >

What is going wrong

Your dungeon generation already connects the rooms with corridors, so the underlying dungeon is generally connected.

The problem is this part of Task 1:

Python

Run

```
if self.grid[r][c] == FLOOR:
    ...
    self.grid[r][c] = TRAP
```

A trap is currently allowed to occupy **any floor cell**.

So imagine the only corridor between the start and the key is:

```
START — floor — floor — floor — KEY
                ↑
                TRAP
```

The dungeon is technically connected, but the player can't traverse that route because stepping on the trap immediately sends them back to the start.

The smallest robust fix

We don't need to regenerate the whole dungeon, change movement, or redesign trap mechanics.

Instead:

1. Find a valid path from **start** → **key**.
2. Find a valid path from **key** → **chest**.
3. Mark every cell on those paths as **protected**.
4. Only place traps on other floor cells.

The traps still exist and still behave exactly as before. We're simply preventing them from occupying the cells that are necessary to win.

And because we're using the existing `grid`, this fits the current architecture nicely.

Minimal code modification

We're going to add **one helper method** to `GameEngine`, then replace only the trap-placement block in `reset()`.

1. Add this helper method

Inside `GameEngine`, add this method **before** `reset()`:

Python

Run

```
def find_path(self, start, target):
    from collections import deque

    queue = deque([start])
    parent = {start: None}

    while queue:
        r, c = queue.popleft()

        if (r, c) == target:
            path = []
            current = target

            while current is not None:
                path.append(current)
                current = parent[current]

            return path

        for dr, dc in [(1, 0), (-1, 0), (0, 1), (0, -1)]:
            nr = r + dr
            nc = c + dc

            if not (0 <= nr < ROWS and 0 <= nc < COLS):
                continue

            if (nr, nc) in parent:
```

```

        continue

    # Anything except a wall can be traversed.
    if self.grid[nr][nc] != WALL:
        parent[(nr, nc)] = (r, c)
        queue.append((nr, nc))

    return []

```

Why BFS?

Because this is a grid, and we only need to answer:

"Can I get from A to B through non-wall cells?"

BFS is perfect for that.

It doesn't modify the dungeon at all.

It simply returns one valid path.

2. Replace the existing trap-generation block

You currently have something like:

Python

Run

```

# Place traps on floor tiles, away from the starting position.
trap_count = 4
placed = 0

while placed < trap_count:
    r = random.randint(0, ROWS - 1)
    c = random.randint(0, COLS - 1)

    if self.grid[r][c] == FLOOR:
        if not (start and start.collidepoint(c, r)):
            self.grid[r][c] = TRAP
            placed += 1

```

Delete only that block.

Replace it with:

Python

Run

```

# Find the key and chest positions.
key_pos = None

```

```
chest_pos = None

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == KEY:
            key_pos = (r, c)
        elif self.grid[r][c] == CHEST:
            chest_pos = (r, c)

# Reserve a guaranteed path from start -> key -> chest.
protected = set()

if start and key_pos and chest_pos:
    start_pos_grid = (start.centery, start.centerx)

    start_cell = (
        start_pos_grid[0],
        start_pos_grid[1]
    )

    # start_cell is in room coordinates, so convert it to row/column.
    start_cell = (start.y, start.x)

    path_to_key = self.find_path(start_cell, key_pos)
    path_to_chest = self.find_path(key_pos, chest_pos)

    protected.update(path_to_key)
    protected.update(path_to_chest)

# Place traps only outside the guaranteed path.
trap_count = 4
placed = 0

trap_candidates = []

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == FLOOR:
            if (r, c) not in protected:
                if not (start and start.collidepoint(c, r)):
                    trap_candidates.append((r, c))

random.shuffle(trap_candidates)

for r, c in trap_candidates:
    if placed >= trap_count:
        break

    self.grid[r][c] = TRAP
    placed += 1
```

One correction I'd make to keep this cleaner

The `start_cell` conversion above can be simplified because `start` is already a `pygame.Rect` whose coordinates are in **grid-cell units**.

So use this directly:

Python

Run

```
start_cell = (start.y, start.x)
```

Therefore the final version of that entire replacement should be:

Python

Run

```
# Find the key and chest positions.
key_pos = None
chest_pos = None

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == KEY:
            key_pos = (r, c)
        elif self.grid[r][c] == CHEST:
            chest_pos = (r, c)

# Reserve a guaranteed path from start -> key -> chest.
protected = set()

if start and key_pos and chest_pos:
    start_cell = (start.y, start.x)

    path_to_key = self.find_path(start_cell, key_pos)
    path_to_chest = self.find_path(key_pos, chest_pos)

    protected.update(path_to_key)
    protected.update(path_to_chest)

# Place traps only outside the guaranteed path.
trap_count = 4
placed = 0

trap_candidates = []

for r in range(ROWS):
    for c in range(COLS):
        if self.grid[r][c] == FLOOR:
            if (r, c) not in protected:
                if not (start and start.collidepoint(c, r)):
                    trap_candidates.append((r, c))
```

```

random.shuffle(trap_candidates)

for r, c in trap_candidates:
    if placed >= trap_count:
        break

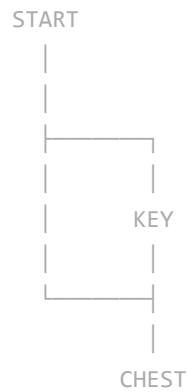
    self.grid[r][c] = TRAP
    placed += 1

```

Use this final version.

What this guarantees

Suppose BFS finds:



Those cells are put into:

Python

```
protected
```

Run

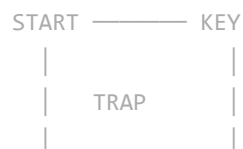
Then trap generation explicitly rejects them:

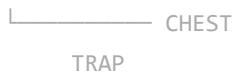
Python

```
if (r, c) not in protected:
```

Run

So you'll get something like:





The player can still encounter traps while exploring, but there is always at least one **trap-free route to the objective**.

Important: your existing trap behavior does NOT change

We haven't touched this:

Python

Run

```
if cell == TRAP:
    self.player.rect.topleft = self.start_pos
    self.status = "Ouch! Trap triggered! Back to START!"
```

So stepping on a trap still does exactly what it did before.

We've only changed **where traps are allowed to spawn**.

Likewise, we're not touching:

- guard movement
- guard collision
- mini-map
- inventory
- key collection
- chest logic
- player movement
- R

So all four existing tasks remain intact.



Testing the fix

Because the bug is random, **one successful run isn't enough**.

Test 1 — Restart repeatedly

Start the game and press:

R

over and over.

I'd do at least **20–30 restarts**.

Each generated dungeon should:

- have traps
- have a key
- have a chest
- have a visible starting position
- have a valid route to the key
- have a valid route from key to chest

Test 2 — Actually win multiple generated dungeons

Don't just inspect the map.

For several random restarts:

1. Start the game.
2. Navigate to the key.
3. Collect it.
4. Navigate to the chest.
5. Confirm:

TREASURE FOUND!

Do this around **10 times**.

This is the strongest practical test because it tests the actual gameplay rather than just the grid.

Test 3 — Confirm traps still exist

On every few generated maps, deliberately step on a trap.

Verify:

```
Trap
↓
Player returns to START
↓
"Ouch! Trap triggered! Back to START!"
```

So we haven't "fixed" the problem by accidentally eliminating traps.

Test 4 — Confirm the protected route really works

This one is useful for the specific bug you discovered.

When you get a suspicious-looking dungeon where a trap is close to a corridor, follow the route to the key.

You should never be **forced** to step onto a trap to reach it.

Then do the same from key → chest.

Test 5 — Verify guard still works

After collecting the key:

1. Touch the guard.
2. Confirm reset to start.
3. Confirm the key remains in inventory.
4. Reach the chest again.

The guaranteed path should still work after the guard collision.

Test 6 — Verify mini-map

On several random restarts, compare:

- actual walls/floors
- key
- chest
- traps
- player position

against the mini-map.

Because we are modifying the same `self.grid`, the mini-map should automatically reflect the protected/trap layout.

One edge case

There is one thing this code intentionally handles gracefully: if a particular generated dungeon doesn't have enough **non-protected floor cells** to place all four traps, it places fewer than four rather than putting a trap on the guaranteed route.

Given your current 20×15 dungeon and eight randomly generated rooms, this should be uncommon, but **playability is more important than forcing exactly four traps**.

If your assignment explicitly requires *exactly* four traps every single generation, then I'd make the next-smallest change be regeneration when fewer than four safe trap positions exist. But I would **not add that complexity unless your rubric explicitly requires exactly four**.



ChatGPT can make mistakes. Check important info.